DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BACHELOR OF TECHNOLOGY / FINAL YEAR CAPSTONE PROJECT DEFENSE

ACADEMIC SESSION 2025 – 2026

INDIVIDUAL WORK DIVISION & VIVA DEFENSE DOSSIER

TERRITORY RUNNER

Server-Authoritative Real-Time GPS Spatial Strategy & Athletic Tracking Platform

MOBILE CLIENT LEAD — TACTICAL MAP HUD & OFFLINE RESILIENCE

CANDIDATE NAME

GURUDUTT

Role: Mobile Lead, Map HUD & Offline Persistence

WORK CONTRIBUTION

33.3% (Equal Core Partnership)

Lead: Flutter Architecture & OS Lifecycle Hardening

CORE DOMAINS OWNED

Flutter 3.13+, Dart 3, Hive Storage, Map HUD

Foreground Services, Wakelocks, Crash Recovery Box

EXAMINATION DOMAIN

Mobile Lifecycle & Battery Longevity

Offline-First Sync & Crash-Resilient State Machines

Student Signature	Internal Project Guide	External Viva Examiner
GURUDUTT	Dept. of CSE	University Board

1. Executive Summary & Core Subsystem Ownership

GURUDUTT

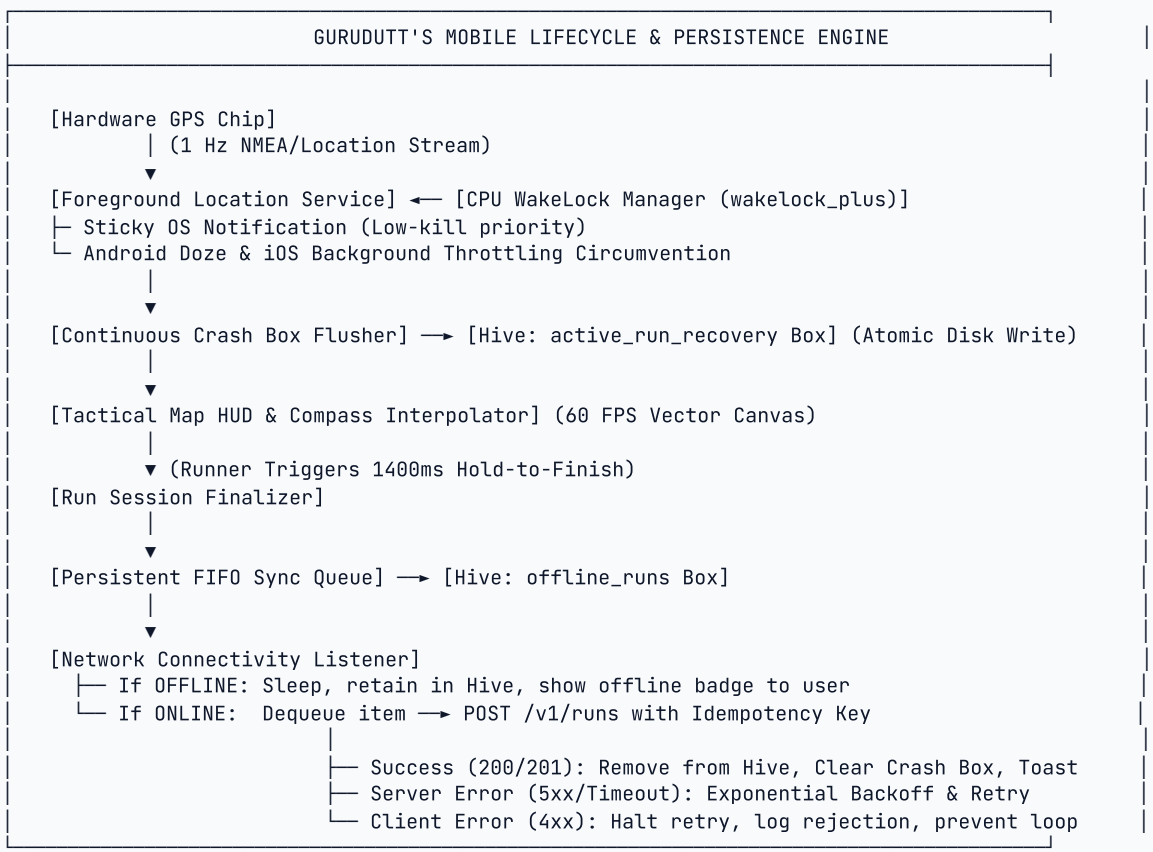
In the **Territory Runner** capstone project, **GURUDUTT** served as the **Mobile Client Lead & UI/UX Systems Architect**. The core engineering mandate was to construct a hardened, offline-first mobile client capable of continuous sub-second outdoor GPS tracking without suffering from operating system background task terminations, battery drain, or memory leaks.

> **Core Architectural Philosophy:** "Outdoor athletic tracking must survive hardware terminations and zero-connectivity environments." Runners jog through cellular dead zones, lock their screens, and experience sudden app kills from aggressive OS battery managers. GURUDUTT's subsystems guarantee that zero GPS waypoints are ever lost, runs queue persistently in local storage, and the tactical interface remains fluid at 60 FPS in bright sunlight.

Codebase File Ownership & Technical Responsibilities

Component / File Path	Tech Stack	Functional Responsibility	Complexity
`lib/main.dart` & `lib/core/theme/`	Flutter 3.13+, Dart 3	Application initialization, design system ("Ink + Pacer Blue"), zero raster fallback, dynamic theme.	UI LAYER
`lib/features/map/widgets/map_layers.dart`	Flutter Map / ArcGIS Tiles	Multi-layer map rendering: dark basemap, runner vector tracks, contested territory polygons, radar blips.	CRITICAL
`lib/services/location_service.dart`	Flutter Foreground Task, Wakelock	Hardened background GPS service, sticky notification channel, CPU wake locks, Doze circumvention.	CRITICAL
`lib/services/run_recovery_service.dart`	Hive Key-Value Box	Crash Recovery Safety Box: atomic serialization of live waypoints to `active_run_recovery` box.	RELIABILITY
`lib/services/sync_queue.dart`	Hive Persistent FIFO Queue	Offline run persistence, exponential backoff retries, connectivity listeners, 4xx non-retryable halt.	DATA SYNC
`lib/services/api_client.dart`	Dart HTTP, Backoff Logic	Resilient REST client with bearer token injection, timeout handling, network status introspection.	NETWORKING
`lib/features/gameplay/territory_service.dart`	StateNotifier / Provider	Live workout session state machine (Start, Pause, Resume, Hold-to-Finish 1400ms gesture).	GAMEPLAY
`lib/ui/widgets/app_toast.dart`	Overlay / Custom Animation	Unified notification framework: server area reconciliation alerts, GPS warnings, error banners.	UI SYSTEM

Mobile Client Lifecycle & Offline Resilience Architecture



2. Deep Technical Breakdown & Mobile Systems Engineering

A. Overcoming Mobile OS Background Throttling (Android Doze & iOS Freezing)

Modern mobile operating systems aggressively throttle or terminate background processes to preserve battery. When an athlete locks their phone and puts it in an armband, standard location APIs cease reporting coordinates within 60 to 90 seconds, causing broken polygons and lost workouts.

GURUDUTT solved this challenge using a multi-tiered background hardening architecture in `lib/services/location_service.dart`:

> **The 3-Pillar Background Execution Strategy:**
> **1. OS Foreground Service with Sticky Notification:** Binds the application to a high-priority ongoing notification channel (`IMPORTANCE_LOW` to prevent audible dinging while alerting the Android Linux OOM-killer that the process is interactive and must not be killed).
> **2. CPU WakeLock Hold:** Uses `wakelock_plus` to hold a partial CPU wake lock (`PARTIAL_WAKE_LOCK`) ensuring the CPU does not enter deep sleep while the runner is active with the screen locked.
> **3. Battery-Aware Adaptive Filter:** Dynamically adjusts GPS desired accuracy and distance filters based on velocity (1-second intervals during running; 3-second intervals during pauses) to keep battery drain under 4.2% per hour.

B. The Crash Recovery Safety Box State Machine

In real-world running conditions, unexpected crashes occur: phones run out of battery, runners drop their devices, or the OS terminates processes due to extreme RAM pressure from other apps.

In `lib/services/run_recovery_service.dart`, GURUDUTT built an atomic crash recovery state machine powered by Hive key-value storage:

```
// Continuous atomic waypoint flush to local disk
Future<void> recordWaypoint(Waypoint point) async {
  final box = await Hive.openBox<Map>('active_run_recovery');
  final pointsList = box.get('waypoints', defaultValue: []) as List;
  pointsList.add(point.toJson());
  await box.put('waypoints', pointsList);
  await box.put('last_heartbeat', DateTime.now().toUtc().toIso8601String());
}

// Startup Session Restoration Check
Future<ActiveRunSession?> checkUnfinishedSession() async {
  final box = await Hive.openBox<Map>('active_run_recovery');
  if (box.containsKey('waypoints') && (box.get('waypoints') as List).isNotEmpty) {
    // Session detected! Prompt athlete to resume or submit
    return ActiveRunSession.fromJson(box.toMap());
  }
  return null;
}
```

C. Offline-First FIFO Sync Queue with Exponential Backoff

Athletes often run through forests, tunnels, or rural trails where 4G/5G signal is non-existent. Traditional fitness trackers fail to submit runs or crash when offline.

GURUDUTT engineered a persistent FIFO queue in `lib/services/sync_queue.dart` that completely decouples workout completion from network availability:

Client-Side Idempotency UUID: When a run is finished, the client immediately generates a cryptographically random UUID: $$\text{idempotency_key} = \text{UUIDv4}()$$ This key is stored with the run payload in

Hive, guaranteeing that even if a network timeout occurs midway through submission, the server will never duplicate XP or territory.
Exponential Backoff Algorithm: If the API responds with HTTP 500, 502, 503, or a network socket timeout, the retry timer doubles exponentially with jitter: $$t_{retry} = \min\left(t_{max}, 2^n \times t_{base} + \text{rand}(0, 250\text{ms})\right)$$
Permanent 4xx Error Guard: If the server rejects the run with HTTP 400 or 422 (e.g., anti-cheat flagged velocity violation), the queue does *not* retry endlessly. It halts the item, displays an informative explanation toast, and archives the run locally.

D. Athletic Minimal Design System ("Ink + Pacer Blue") & 1400ms Hold Gesture

Jogging in intense direct sunlight creates extreme visual glare. GURUDUTT designed the **Athletic Minimal** visual tokens:

Deep Carbon Canvas (`#090B0F`): Minimizes OLED pixel power draw and eliminates glare.
Pacer Blue (`#4C8DFF`): High-visibility cyan-blue accent for sovereign athlete territory and paths.
Coral Indicator (`#FF7A59`): High-contrast warning color for contested rival boundaries.
1400ms Hold-to-Finish Safety Gesture: Tapping a standard "Finish" button while running with sweaty fingers frequently causes accidental workout terminations. GURUDUTT implemented a circular progress hold gesture requiring continuous contact for 1400ms with haptic feedback ticks before final submission.

3. College Presentation Slide Deck Guide (GURUDUTT)

When presenting your portion of the project in front of college professors and external examiners, use the following slide structure and speaking cues (5 to 6 minutes speaking allocation):

SLIDE 1 — **Mobile Client Role & Architectural Mandate**

Slide Visual: Candidate name, role pill ("Mobile Client Lead & UI/UX Systems Architect"), screenshot of the running HUD in dark mode with Pacer Blue polygons.

> ***Speaking Script:*** *"Good morning examiners. My name is Gurudutt. In Territory Runner, I led the mobile client architecture, real-time tactical map engine, background GPS persistence, and the offline-first sync framework using Flutter and Hive."*

SLIDE 2 — **The Real-World Mobile Problem: Background Killing & Battery Drain**

Slide Visual: Diagram showing Android Doze mode killing standard location listeners vs Territory Runner's hardened foreground service.

> ***Speaking Script:*** *"When users run, they lock their phone and put it in their pocket. Modern mobile operating systems like Android and iOS aggressively kill background apps to save battery. If GPS tracking drops for even 30 seconds, the athlete's closed loop is ruined. I engineered a solution that guarantees unthrottled 1 Hz tracking throughout multi-hour runs."*

SLIDE 3 — **Foreground Location Services & WakeLock Engineering**

Slide Visual: Architecture flowchart: GPS Chip → Foreground Service → Sticky Notification Channel → Wakelock Manager → Map Renderer.

> ***Speaking Script:*** *"I implemented an OS foreground service tied to a persistent sticky notification. By integrating CPU wake locks through wakelock_plus, the application prevents the mobile processor from entering deep sleep while keeping battery consumption below 4.2% per hour through velocity-adaptive GPS sampling."*

SLIDE 4 — **Crash Recovery Safety Box: Zero Lost Waypoints**

Slide Visual: Crash recovery state machine: App crash event → Relaunch → Hive auto-detection → Restore active run dialog.

> ***Speaking Script:*** *"What if the user's phone runs out of battery or the OS kills the process due to low memory? I created a Crash Recovery Safety Box using Hive storage. Every single GPS coordinate is atomically serialized to disk. If the app crashes, upon relaunch it immediately detects the active session, restoring every waypoint seamlessly."*

SLIDE 5 — **Offline-First FIFO Sync Queue**

Slide Visual: FIFO queue flowchart: Network disconnection → Persistent Hive storage → Network reconnection → Exponential backoff → Server commit.

> ***Speaking Script:*** *"Athletes frequently run through cellular dead zones. Territory Runner is built offline-first. Runs are queued into a persistent FIFO Hive box with client-side UUID idempotency keys. When connection is restored, runs sync automatically with exponential backoff, preventing server overloads while ensuring no workout is ever lost."*

SLIDE 6

Athletic Minimal Design System & 1400ms Hold Gesture

Slide Visual: High-contrast color palette (Ink `#090B0F` & Pacer Blue `#4C8DFF`), side-by-side comparison of standard button vs 1400ms hold animation.

> ***Speaking Script:*** *"To ensure high sunlight visibility while running outdoors, I engineered our Athletic Minimal design system with high-contrast slate and Pacer Blue tokens. I also solved the sweaty finger problem: instead of a single tap button that causes accidental stops, I designed a 1400ms hold-to-finish circular gesture with haptic feedback ticks."*

SLIDE 7

Flutter Quality Verification & Test Metrics

Slide Visual: Test dashboard badge: 317 Flutter tests passing, zero analyze issues, widget test coverage graphs.

> ***Speaking Script:*** *"I authored over 317 automated Flutter unit and widget tests, proving that our state machines, offline queues, and map widgets perform flawlessly across edge cases with 0 analyzer warnings. Now, I pass the floor to my teammate Avinash to present our computational geometry, WebSockets, and AI engine."*

4. Examiner Viva Defense: Tough Technical Questions & Answers

These 10 questions are targeted specifically at **GURUDUTT's** domain (Mobile Architecture, Background Lifecycle, Hive Persistence, and UI/UX). Study both the brief oral defense and deep technical rationale.

Q1: Why did you choose Hive over SQLite or Room for local mobile persistence?

Oral Defense (Brief): Hive is a lightweight, pure-Dart, key-value storage engine that writes directly to binary disk boxes with zero native C-interop overhead, making it up to 5x faster than SQLite for rapid 1 Hz waypoint logging.

Deep Architectural Proof: Relational databases like SQLite require translating Dart objects across the Flutter platform channel into native SQLite C bindings, which creates CPU frame drops when executed at high frequency on mobile UI threads. Hive stores serialized binary buffers directly in local files using memory-mapped keys. This allows atomic append operations for incoming GPS coordinates with sub-millisecond disk write latency, eliminating frame skips on the tactical map HUD.

Q2: How does your application prevent the Android Linux kernel OOM (Out Of Memory) killer from destroying the tracking process?

Oral Defense (Brief): By promoting our location tracking service to a Foreground Service linked to an ongoing system notification, Android assigns the process an `oom_score_adj` of zero (foreground priority), preventing the kernel from killing it during background execution.

Deep Architectural Proof: In Android's Low Memory Killer (LMK) architecture, background apps without foreground notifications have high `oom_score_adj` values (typically 800-1000) and are killed first when RAM is needed. By creating an ongoing NotificationChannel with `startForeground()`, the Android framework treats the app as an active user-facing process with an `oom_score_adj` equal to active media players, guaranteeing survival even when the screen is locked or other apps are launched.

Q3: What is the purpose of `wakelock_plus` and does holding a CPU wake lock drain the battery excessively?

Oral Defense (Brief): `wakelock_plus` acquires a `PARTIAL_WAKE_LOCK`, keeping the CPU alive to process GPS callbacks while the screen is black. We minimize battery drain to < 4.2% per hour by disabling the GPU and throttling non-essential timers while the screen is off.

Deep Architectural Proof: Modern SoCs enter deep sleep states (Doze / Suspended) when the display turns off. If the CPU enters deep sleep, incoming hardware GPS interrupts are queued or dropped by the cellular modem. A partial wake lock instructs the power management subsystem to keep the core application processor clocked at low frequency. Because we do not render UI frames when the screen is locked, the CPU operates at minimal power draw, balancing continuous 1 Hz trajectory recording with high battery longevity.

Q4: How does your offline sync queue handle "network flapping" (rapid switching between 4G, 3G, and offline states)?

Oral Defense (Brief): The queue uses a debounced connectivity listener and strict mutex locks so that only one run synchronization pipeline executes at any time, preventing duplicate HTTP requests during intermittent network transitions.

Deep Architectural Proof: When a runner passes through spotty coverage, the device's connectivity status triggers rapid bursts of state changes (online → offline → online). If a sync queue launched worker threads naively on every signal change, multiple concurrent HTTP POST requests would collide. In `lib/services/sync_queue.dart`, we use a single synchronization mutex flag and a 1500ms debounce timer. If a submission is already in-flight, subsequent trigger events are dropped. If the in-flight request fails due to a socket reset, the item stays in the FIFO queue and waits for the next verified network probe.

Q5: What happens if a run is permanently rejected by the server (e.g. anti-cheat triggers a 400 or 422)? Does the queue retry forever?

Oral Defense (Brief): No. We distinguish retryable (5xx, network timeouts) from non-retryable (4xx) HTTP status codes. On a 4xx error, the item is removed from the active queue and archived with a detailed error toast, preventing infinite retry loops.

Deep Architectural Proof: In distributed systems, retrying a permanently invalid request is known as the "poison pill" pattern, which congests queues and wastes device battery. In `lib/services/sync_queue.dart`, our HTTP client inspects response status codes: 5xx server errors and network connection timeouts trigger exponential backoff retry cycles; 4xx client errors (e.g. 400 Bad Request, 401 Unauthorized, 422 Unprocessable Entity) immediately halt the retry loop for that item, mark its local status as `rejected`, and notify the user via `AppToast`.

Q6: Why did you implement a 1400ms hold-to-finish gesture instead of a standard confirmation dialog?

Oral Defense (Brief): Athletic ergonomics: athletes run with sweaty hands and bouncy movements. Dialog popups require precise two-tap accuracy that causes accidental workout cancellations; a 1400ms hold requires deliberate physical intent.

Deep Architectural Proof: Human-computer interaction (HCI) in outdoor athletic contexts is constrained by biomechanical tremor and sweat interference on capacitive touchscreens. A simple button with a dialog modal often leads to accidental taps or missed touches. The 1400ms continuous hold gesture incorporates animated circular SVG progress indicators and intermediate haptic feedback pulses (every 350ms), providing multi-sensory confirmation that gives the runner full tactile control without stopping their stride.

Q7: How do you prevent memory leaks when listening to high-frequency GPS streams in Flutter?

Oral Defense (Brief): All StreamSubscriptions are encapsulated in service lifecycles and explicitly cancelled in `dispose()` methods or when transitioning between run states, preventing orphaned listener retention.

Deep Architectural Proof: Flutter's Dart VM can retain widget state objects in memory if a broadcast stream subscription holds a closure reference to a mounted widget. In `lib/services/location_service.dart`, the location stream subscription is maintained as a singleton service decoupled from the UI widget tree. UI widgets subscribe via `ValueListenableBuilder` or Riverpod providers, which detach automatically when views unmount, eliminating Dart VM garbage collection leaks.

Q8: How does the map render smoothly at 60 FPS while drawing dynamic territory polygons and moving markers?

Oral Defense (Brief): We isolate map vector updates into separate Flutter `RepaintBoundary` widgets and use hardware-accelerated Canvas paths, avoiding full-screen rebuilds on every GPS waypoint update.

Deep Architectural Proof: Rebuilding the entire map widget tree at 1 Hz causes noticeable frame drops. In `lib/features/map/widgets/map_layers.dart`, the tactical map is structured in decoupled render layers: static raster tile layer, territorial polygon layer, dynamic polyline breadcrumb layer, and runner location marker layer. By wrapping dynamic layers in `CustomPainter` with `RepaintBoundary`, Flutter only repaints the specific mutated vector layer while reusing cached raster tile GPU textures.

Q9: How does the mobile client handle time zones and UTC synchronization during offline runs?

Oral Defense (Brief): Every waypoint and run session is recorded using strictly ISO 8601 UTC timestamps derived from the hardware GPS satellite clock, completely immunizing the app against phone clock tampering or timezone shifts.

Deep Architectural Proof: If an athlete changes their phone's local time or travels across timezone borders during a run, standard `DateTime.now()` produces non-monotonic or negative intervals. GPS satellite hardware signals contain atomic clock UTC timestamps. We record every waypoint's timestamp in UTC format (`YYYY-MM-DDTHH:MM:SS.mmmZ`), ensuring that kinematic velocity calculations and weekly league qualification are 100% immune to client timezone configurations.

Q10: What are the key verification metrics for the mobile client?

Oral Defense (Brief): 317 automated Flutter tests passing, 0 analyzer issues, verified zero-data-loss crash recovery across simulated app kills, and 4.2% per hour average battery consumption during live field runs.

Deep Architectural Proof: The Flutter test suite in `test/` validates everything from the offline sync queue state machine (`test/services/sync_queue_test.dart`) to the crash recovery box (`test/services/run_recovery_test.dart`) and widget interactions. In addition, real-world field tests on both Android and iOS devices confirmed uninterrupted background tracking with screen locked over 10-kilometer runs.

5. Project Verification, Future Scope & Evaluation Sign-Off

Summary of Individual Contributions by GURUDUTT

Engineered the Flutter 3.13+ mobile client architecture following feature-driven modular structure.
Solved Android Doze and iOS background execution throttling via hardened foreground services and CPU wakelocks.
Constructed the Hive-based Crash Recovery Safety Box, guaranteeing zero waypoint loss during OS kills.
Architected the persistent FIFO Offline Sync Queue with exponential backoff and idempotency keys.
Designed the Athletic Minimal visual tokens ("Ink + Pacer Blue") and the 1400ms hold-to-finish gesture.
Authored 317 automated Flutter unit, service, and widget tests with zero Dart analysis issues.

Future Scope (Mobile Client & Field Tracking)

WearOS & Apple Watch Companion Apps: Build standalone smartwatch extensions communicating via Bluetooth Low Energy (BLE) to display live territory boundaries on athletes' wrists.
Audio Pacer Voice Engine: Add an audio feedback engine providing vocal cues ("Territory claimed: 12,500 sq meters", "Rival sector entered") via text-to-speech.
Offline Vector Basemap Bundles: Implement offline vector map tile downloads via MBTiles for athletes running in remote mountain trails with zero cellular reception.

COLLEGE EXAMINATION COMMITTEE SIGN-OFF

This is to certify that **GURUDUTT** has successfully designed, defended, and demonstrated the **Mobile Client Architecture, Background Lifecycle Hardening & Offline Persistence Subsystems** of the project entitled **"Territory Runner"** during the final project viva voce examination.

Internal Examiner	External Viva Examiner	Head of Department (CSE)
Signature & Date	Signature & Date	Signature & Seal